Multi-Step LLM Agent: League of Legends Dynamic Matchup Strategist

By – Saksham Bali (U20230061)

Written Report

1. Problem Statement

League of Legends is a highly dynamic competitive game where champions, items, and matchups change frequently due to frequent balance patches. Players often need quick strategic advice before entering a ranked game, especially during the loading screen when they have only a few minutes to prepare. A single-prompt Large Language Model performs poorly for this task because it tries to handle parsing, information retrieval, strategic reasoning, and final formatting at once. This often produces outdated builds, hallucinated items, or generic advice such as "farm safely" without real matchup-specific depth.

This project solves that problem by building a multi-step LLM agent that acts as a high-ELO loading screen coach. The agent takes the player's champion, role, support pairing, lane opponent, enemy support, and jungle threat, then generates a personalized strategy report including matchup difficulty, itemization, power spikes, minute-by-minute tactical planning, and win conditions.

This task benefits from multi-step chaining because each stage depends directly on the previous one. The system must first parse raw user input into structured game-state data, then retrieve live patch information using an external tool, then reason about matchup strategy using both inputs, and finally format the result into a usable report. If one step fails, the next step is affected, making this a true chained pipeline rather than independent prompts. This directly satisfies the assignment requirement for explicit dependency between steps.

2. Chain Design

The system uses a shared Python dictionary called state, where each step reads from and writes to the same object. This allows complete traceability during execution and makes debugging easier during the live demo.

Step 1: Parse User Input

The first LLM call converts natural language input into strict JSON containing user role, champion, lane opponent, ally support, enemy support, jungle information, and damage type (AP/AD/MIXED). This step is necessary because tool calls require structured parameters.

A major issue during testing occurred when Lux bot lane was treated as a normal ADC and received AD crit builds. To fix this, explicit damage_type extraction was added so later steps use champion scaling instead of role assumptions.

Step 2: Tool Call – Web Search

DuckDuckGo Search (DDGS) is used to retrieve current patch build information and matchup guides using two targeted search queries. Results are stored in the shared state.

This step is essential because League of Legends changes too often for static LLM knowledge to remain reliable. If search fails, the system switches to training-data fallback instead of breaking the chain.

Step 3: Matchup Analysis

The second LLM reasoning step combines parsed champion data and live search results to generate power spikes, build recommendations, matchup difficulty, and win conditions.

Strict build constraints were added to prevent AP champions from receiving AD builds and to stop hallucinated removed items like Galeforce. This improved strategic correctness significantly.

Step 4: Tactical Timeline

The third LLM call creates a minute-by-minute game plan from minute 1 to 20, including wave management, trading patterns, jungle tracking, objectives, and rotations.

Prompt improvements were added to prevent repetitive outputs and force real macro events such as Scuttle timing, first Dragon, turret plates falling at 14 minutes, and post-laning rotations. Unknown junglers are treated as high threat instead of "none."

Step 5: Final Formatting

The final LLM call converts the populated state into a polished Markdown strategy report containing matchup overview, recommended build, tactical timeline table, win condition, and data source note. The report is saved as a file so it can be directly used by the player.

3. Tool Integration

The external tool used is DuckDuckGo Search through the DDGS Python library. This satisfies the mandatory tool-call requirement because current patch builds and matchup statistics cannot be reliably generated from model training alone.

The parsed state from Step 1 is used to generate specific search queries for champion builds and matchup advice from sources such as U.GG, Lolalytics, and Mobafire. The returned search results are stored inside the shared state and passed into Step 3 for reasoning.

If search results are incomplete or unavailable, the system explicitly records training-data fallback instead of hiding the limitation. This makes tool failure visible and easier to explain during evaluation.

4. Limitations

The main limitation is search quality. DDGS often returns only short snippets rather than full pages, so the model may not receive enough context for complete builds. This can cause fallback to training data and occasional hallucinations such as removed items or weak recommendations.

Another limitation is that strategy quality is harder to verify than factual tasks because League of Legends advice can be subjective depending on player rank and playstyle.

Testing revealed failures such as Lux receiving ADC builds and repetitive "farm safely" timelines from minutes 12–20. These were fixed using stricter prompt constraints, better intermediate state design, and AP/AD build gates, but strategic hallucination can still occur.

5. Reflection

If given more time, I would replace snippet-based search with stronger structured APIs such as Riot API or Riot Data Dragon for better build validation and reliability. I would also add a validation step after Step 1 to verify champion names before searching.

Another improvement would be adding a second LLM review step where the generated report is critiqued before final output, creating a self-correction loop for weak builds or repetitive advice.

The biggest lesson from this project was that prompt design matters more than model size. The initial pipeline worked technically but produced weak strategy because prompts were too generic. Adding

explicit constraints for AP/AD itemization, removed item prevention, and unique macro decisions improved output quality dramatically.

A separate Prompt Design Appendix has been attached containing the full prompts for each LLM step, prompt iteration examples, testing failures, and explanations of how prompt changes improved the system, as required by the assignment.